

Question 1: Forward Propagation & Structure

Q: How do Recurrent Neural Networks (RNNs) handle sequential input data of varying lengths during forward propagation?

RNN ফরওয়ার্ড প্রোপাগেশনের সময় সিকোয়েন্সের প্রতিটা উপাদানকে সময়-ধাপ অনুযায়ী (Step-by-Step / Recurrent Loop) প্রসেস করে, যেখানে স্থির ওয়েট (Shared Weights) ব্যবহার করে যেকোনো দৈর্ঘ্যের ডাটা হ্যান্ডেল করা যায়।

আর ব্যাচ হিসেবে প্রসেস করার সময়:

1. **Padding:** ছোট ডাটাগুলোকে জিরো (0) বসিয়ে সমান দৈর্ঘ্যের করা হয়।
2. **Masking (Dynamic Unrolling):** প্যাডিং অংশগুলোকে হিসাব থেকে বাদ দিয়ে শুধু আসল ডাটার অংশটুকু ফরওয়ার্ড পাস করা হয়।

Question 2: Shared Parameters

Q: When an RNN is unrolled across multiple time steps, what happens to the weights and biases?

RNN যখন একাধিক টাইম-স্টেপে আনরোল (Unroll) করা হয়, তখন এর ওয়েট (**Weights**) এবং বায়াস (**Biases**) সম্পূর্ণ অপরিবর্তিত থাকে—অর্থাৎ এগুলো সব টাইম-স্টেপে শেয়ার বা পুনঃব্যবহার করা হয় (**Shared Parameters**)।

মূল বিষয়সমূহ:

- ওয়েট শেয়ারিং (**Weight Sharing**): মডেলের প্যারামিটার ম্যাট্রিক্সসমূহ— W_{xh} (Input-to-Hidden), W_{hh} (Hidden-to-Hidden), W_{hy} (Hidden-to-Output)—এবং বায়াস ভেক্টরসমূহ (b_h, b_y) প্রথম সময়-ধাপ ($t = 1$) থেকে শুরু করে শেষ সময়-ধাপ ($t = T$) পর্যন্ত একই থাকে।
-
- প্যারামিটারের সংখ্যা নির্দিষ্ট থাকা: ওয়েট শেয়ারিংয়ের কারণে সিকোয়েন্স যত দীর্ঘই হোক না কেন, লার্নেবল প্যারামিটারের মোট সংখ্যা বাড়বে না বা নতুন কোনো ওয়েট যুক্ত হবে না।
-
- ব্যাকপ্রোপাগেশন (**BPTT**):
- প্রশিক্ষণের সময় ব্যাকপ্রোপাগেশন ৩ টাইম (BPTT) পদ্ধতিতে প্রতিটি টাইম-স্টেপের লস থেকে গ্র্যাডিয়েন্ট হিসাব করা হয় এবং সবশেষে শেয়ার্ড ওয়েটগুলোর জন্য মোট গ্র্যাডিয়েন্ট যোগ (Sum) বা গড় (Average) করে একবারে ওয়েট আপডেট করা হয়।

Question 3: Exploding Gradient Problem

Q: During backpropagation over many time steps, what causes the gradient to "explode," and how does this affect training?

১. গ্র্যাডিয়েন্ট এক্সপ্লোড হওয়ার কারণ

1. পুনরাবৃত্তিমূলক ম্যাট্রিক্স গুণন (**Chain Rule**): BPTT চলাকালীন সময়ের সাথে সাথে (যেমন T সময়-ধাপ থেকে t সময়-ধাপে) গ্র্যাডিয়েন্ট গণনা করার জন্য চেইন রুল (Chain Rule) ব্যবহার করা হয়। এতে মধ্যবর্তী সব টাইম-স্টেপের জ্যাকোবিয়ান ম্যাট্রিক্সগুলো পর পর গুণ হতে থাকে:

$$2. \frac{\partial h_t}{\partial h_t} = \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}}$$

3. আইগেনভ্যালু (**Eigenvalue**) ১-এর বেশি হওয়া: রিকারেন্ট ওয়েট ম্যাট্রিক্স W_{hh} -এর সবচেয়ে বড় আইগেনভ্যালু যদি 1-এর বেশি হয় (বা ওয়েটগুলোর মান তুলনামূলক বড় হয়), তবে সিকোয়েন্সের দৈর্ঘ্য বাড়ার সাথে সাথে W_{hh}^{T-t} হিসাবের কারণে গ্র্যাডিয়েন্টের মান সূচকীয়ভাবে (**Exponentially**) বাড়তে থাকে।

২. ট্রেনিংয়ে এর প্রভাব

- অস্থিতিশীল প্যারামিটার আপডেট: অত্যন্ত বড় গ্র্যাডিয়েন্টের কারণে মডেলের ওয়েট এক লাফে অনেক বেশি পরিবর্তন হয়ে যায়।
- লস ডাইভারজেন্স (**Loss Divergence & NaN/Inf**): মডেলের লস স্মুথলি কমে না এসে হঠাৎ অনেক বেড়ে যায় বা লাফালাফি করে। অনেক সময় PyTorch/TensorFlow-এ লস বা ওয়েটের মান **NaN** (Not a Number) অথবা **Inf** (Infinity) দেখায়।
- লার্নিং নষ্ট হয়ে যাওয়া: অপটিমাইজার লস সারফেসে এতো বড় লাফ দেয় যে, মডেল আগে যা কিছু শিখেছিল তা সম্পূর্ণ মুছে যায় এবং ট্রেনিং পুরোপুরি ব্যর্থ হয়।

৩. সমাধানের উপায়

- গ্র্যাডিয়েন্ট ক্লিপিং (**Gradient Clipping**): গ্র্যাডিয়েন্টের মান বা নরম (Norm) একটি নির্দিষ্ট থ্রেশহোল্ড (যেমন M) অতিক্রম করলে সেটিকে জোরপূর্বক কেটে ছোট করে দেওয়া হয়:
- সঠিক ওয়েট ইনিশিয়ালাইজেশন: Orthogonal বা Xavier/Glorot ইনিশিয়ালাইজেশন ব্যবহার করা।
- গেটেড আর্কিটেকচার: প্লেইন RNN-এর বদলে LSTM বা GRU আর্কিটেকচার ব্যবহার করা।

Question 4: Vanishing Gradient Problem

Q: What causes the "vanishing gradient" problem during backpropagation through time, and what issue does it create?

১. ভ্যানিশিং গ্র্যাডিয়েন্টের কারণ (What Causes It): BPTT-এর সময় সিকোয়েন্সের অনেকগুলো টাইম-স্টেপ জুড়ে যখন বার বার ছোট মান যুক্ত ওয়েট ম্যাট্রিক্স (W_{hh}) এবং $\tanh$ বা sigmoid এক্টিভেশন ফাংশন (যাদের ডেরিভেটিভের মান 1-এর কম) গুণ করা হয়, তখন চেইন রুল (Chain Rule)-এর কারণে গ্র্যাডিয়েন্টের মান সূচকীয়ভাবে কমে শূন্যের (0) কাছাকাছি চলে যায়।

২. এর ফলে তৈরি হওয়া সমস্যা (What Issue It Creates)

- দীর্ঘমেয়াদী মেমোরি হারিয়ে ফেলা (Loss of Long-Term Memory): অতীতের টাইম-স্টেপগুলোতে গ্র্যাডিয়েন্ট সিগন্যাল না পৌঁছানোর ফলে মডেল শুরুর দিকের কোনো তথ্য মনে রাখতে পারে না।
- লং-রেঞ্জ কনটেক্সটে ব্যর্থতা: বড় সিকোয়েন্সের ক্ষেত্রে টেক্সট বা ডাটার শুরুর দিকের তথ্যের সাথে শেষের দিকের তথ্যের সম্পর্ক রক্ষা করতে পারে না।
- ট্রেনিং থমে যাওয়া: নেটওয়ার্কের শুরুর দিকের লেয়ার বা সময়-ধাপগুলোর লার্নিং বা ওয়েট আপডেট পুরোপুরি বন্ধ হয়ে যায়।

Question 5: Sequential Information Flow

Q: In forward propagation, why are inputs fed sequentially from oldest to newest data points?

- Causality & Temporal Order:** বাস্তব জীবনে অতীত ঘটনা বর্তমানের ওপর প্রভাব ফেলে, কিন্তু ভবিষ্যৎ অতীতের ওপর ফেলে না। যেমন—একটি বাক্যে "আমি ভাত ..." শোনার পর "খাবো" শব্দটি আসে। RNN ঠিক মানব মস্তিষ্কের মতো অতীত তথ্য থেকে ধারণা নিয়ে বর্তমান সময়ের প্রেডিকশন তৈরি করে।
- Sequential Memory/Hidden State Update:** RNN-এর মেমোরি বা হিডেন স্টেট h_t তৈরি হয় তার পূর্ববর্তী হিডেন স্টেট h_{t-1} এবং বর্তমান ইনপুট x_t -এর ওপর ভিত্তি করে ($h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$)। পুরনো ডাটা থেকে আগে প্রসেস না করলে h_{t-1} -এ কোনো কনটেক্সট বা মেমোরি জমা হবে না, ফলে নতুন ডাটা প্রসেস করার মতো প্রয়োজনীয় তথ্য মডেলের কাছে থাকবে না।
- Autoregressive Flow:** টেক্সট জেনারেশন বা টাইম-সিরিজ প্রেডিকশনে বর্তমান সময়ের আউটপুট (y_t) এর ওপর ভিত্তি করে পরবর্তী সময়-ধাপের ইনপুট চালিত হতে পারে। তাই কালানুক্রমিকভাবে (Chronological Order) ডাটা পাস করা অপরিহার্য।